PRACTICAL TEST

Software Engineer Practical Test — Structured Content API & Editor

Build a small full-stack application: a web API that models a simplified version of Author-it's content domain, and a front end that lets an author browse and edit that content.

ROLE	ESTIMATED TIME	QUESTIONS
Software Engineer	4–6 hours	osmar.silva@author-it.com

Overview

The exercise is grounded in how Author-it structures content. A library holds reusable objects: Topics hold the content, and Books define structure and sequence by referencing Topics. A Book never owns or copies a Topic, so the same Topic can appear in many Books, or more than once in the same Book.

The goal is not to build Author-it. We want to see how you model a reference-based, reuse-oriented domain, how you design the API contract, and how you build an editing experience on top of it.

- **Part A — API:** Topics, Books, the references between them, and a reuse report.
- **Part B — Front end:** a working UI that views and edits content through your API.

> We would rather see a less complete but well-considered submission than a complete but rushed one. Code clarity, design decisions and your notes on trade-offs matter as much as working features.

Time guidance

PART	SUGGESTED TIME
Part A — API, including tests	~2 hours
Part B — Front end	2–3 hours
README	~30 minutes

Please stop at 6 hours. Whatever is unfinished, list it in your README with a line on how you would approach it. We score your notes, not the gap. Stretch goals are listed at the end; leaving them out costs you nothing.

Technology

Back end. Any language or framework. We recommend .NET 10+ with C#, as that is what we use, but we are happy to review other languages.

Front end. We prefer React with TypeScript. Vue, Angular, Svelte or another modern framework is also fine — tell us in your README why you chose it. Data fetching, state management, routing and styling are your call. For editing topic bodies you may use a plain textarea, Markdown, or a rich-text editor library such as TipTap, Lexical or Slate.

Storage. In-memory storage is fine throughout. No database is required.

Running it. The project should start with one or two documented commands, for example, one per app. Docker is not required. Please document the ports and any CORS setup.

Third-party libraries are fine on either side. Use of LLMs and AI coding tools is expected and welcome. We are interested in the decisions you make and whether the result reflects real understanding of the domain, so expect to walk us through your design in a follow-up conversation.

> QUESTIONS
>
> Questions are welcome at any time at osmar.silva@author-it.com. There is no penalty for asking.

PART A

API

Part A is ten endpoints over three concepts: Topics, Books, and the references that place a Topic inside a Book.

STAGE	METHOD	PATH	PURPOSE
1	GET	/topics	List all topics
1	GET	/topics/{id}	Get one topic
1	POST	/topics	Create a topic
1	PUT	/topics/{id}	Update a topic, with conflict detection
2	GET	/books	List all books (no contents)
2	GET	/books/{id}	Get one book with its ordered contents
2	POST	/books	Create a book
3	POST	/books/{id}/references	Add a topic reference to a book
3	DELETE	/books/{id}/references/{referenceId}	Remove one reference
4	GET	/topics/{id}/usage	Where a topic is used, across all books

Use HTTP verbs and status codes as you would in production, and return errors in one consistent shape. Where more than one status code is defensible, pick one and say why in your README.

Stage 1 — Topics

A Topic has:

- `id` — unique
- `title`
- `body` — plain text
- `tags` — zero or more strings
- `created` and `updated` — timestamps
- `version` — changes on every successful update

`POST /topics` returns the created topic in the same shape as `GET /topics/{id}`.

Validation. Apply these rules on create and update, and return errors that identify the field at fault so the front end can show them in place:

- `title` is required and at most 200 characters.

- Each tag is non-empty and at most 50 characters.
- Tags are unique within a topic, ignoring case.

Conflict detection. `PUT /topics/{id}` must reject a write based on a stale copy. The client sends the version it last read, by header or body (your choice). If the topic has changed since, reject the update with 409 or 412 and do not apply it.

Seed data. Seed these topics. Give each a short placeholder body of a sentence or two.

ID	TITLE	TAGS
1	Introduction to Author-it	overview, getting-started
2	Installing the Desktop Client	installation, windows, desktop
3	Connecting to a Library	setup, library, configuration
4	Creating Your First Topic	authoring, topics, beginner
5	Publishing to PDF	publishing, output, pdf

Stage 2 — Books

A Book has an `id`, a `title`, an optional `description` and a `created` timestamp. A new Book has no contents.

`GET /books` returns basic properties only. `GET /books/{id}` also returns the Book's contents: an ordered list of entries.

The contents model. A Book's contents is a flat, ordered sequence of references. Each entry has:

- `referenceId` — identifies this entry, not the topic
- `topicId` and `topicTitle`
- `level` — indent depth: 1 is a chapter, 2 a section inside it, and so on
- `position` — the entry's place in the book's sequence, starting at 1

The hierarchy comes from reading the sequence in order: an entry belongs under the nearest earlier entry with a lower level. Two rules keep it valid: the first entry is at level 1, and an entry's level is at most one deeper than the entry before it.

We chose this model so you can spend your time on reuse and editing rather than tree mechanics. If you would have modelled it differently, tell us in your README.

Seed data. Seed these three books. Note that Topic 3 appears twice in the Administrator Guide.

BOOK	POSITION	TOPIC	LEVEL
1 — Getting Started Guide	1	1 — Introduction to Author-it	1
	2	2 — Installing the Desktop Client	1
	3	3 — Connecting to a Library	2
	4	4 — Creating Your First Topic	1
2 — Publishing Handbook	1	1 — Introduction to Author-it	1
	2	5 — Publishing to PDF	1
	3	4 — Creating Your First Topic	2
3 — Administrator Guide	1	2 — Installing the Desktop Client	1
	2	3 — Connecting to a Library	2
	3	5 — Publishing to PDF	1
	4	3 — Connecting to a Library	2

Stage 3 — Adding and removing references

`POST /books/{id}/references` adds an entry. The request includes `topicId`, `level` and an optional `position`. Inserting at a position shifts later entries down by one; omitting `position` appends to the end.

- The same topic may be referenced any number of times, in different books or in the same book. This is intentional and reflects how Author-it works.
- If the topic does not exist, return an appropriate error.
- If the level breaks the rules above, return a validation error.

`DELETE /books/{id}/references/{referenceId}` removes that one entry and closes the gap in positions. It never deletes the topic. In your README, say what happens to entries that were indented beneath the removed one.

Stage 4 — Reuse report

`GET /topics/{id}/usage` shows where a topic is used. The response includes:

- the topic's `id` and `title`
- each book that references it, with the `referenceId`, `level` and `position` of every reference in that book
- `totalReferences` — references across all books combined
- `bookCount` — distinct books that use it

With the seed data, Topic 3 should report 3 references across 2 books. This is the point of the reference model: reuse is visible without any content being duplicated.

PART B

Front end

Part B is a UI with three screens that lets an author work with the library through your API. It must call the real API, not mocks. Desktop layout only. Visual polish matters less than clear structure, sensible loading and error states, and a smooth editing flow.

Stage 5 — Topic library

- List all topics with title, tags and last-updated time.
- Filter by title or tag. Filtering in the browser is fine.
- Create a new topic and land in the editor for it.
- Handle loading, empty and error states.

Stage 6 — Topic editor

This is the heart of Part B.

- Edit a topic's title, body and tags, and save through the API.
- Show validation errors from the API next to the relevant field.
- Show clearly when there are unsaved changes.
- **Reuse warning.** Use `GET /topics/{id}/usage` to show which books contain this topic. If it is used in more than one book, make it clear that saving changes the content in all of them.
- **Edit conflicts.** If a save is rejected because the topic changed since it was loaded, tell the author and keep their edits in the form so nothing they typed is lost. Offering a way to reload the latest version is enough; a merge or diff view is not expected. Two browser tabs on the same topic is a fine way to demonstrate this.

Stage 7 — Book outline

Books are seeded, so creating books in the UI is not required.

- List books, and open one.
- Show the book's contents as an indented outline, in order.
- Add a reference: pick an existing topic, then choose its position and level. Simple inputs are fine; drag and drop is not expected.
- Remove a reference.
- Open any entry in the topic editor from the outline.

After any edit, the screens should reflect the server's current state. For example, a renamed topic should show its new title in every book outline without a manual page refresh.

Testing

Write three to five focused API tests on the behaviour that matters most. Good candidates:

- Adding a reference to a topic that does not exist is rejected.
- The usage report counts references correctly across books, including repeats within one book.
- An update based on a stale version is rejected and changes nothing.

A few well-chosen tests tell us more than broad coverage of trivial cases. Front-end tests are optional.

Stretch goals

These are optional. Leaving them out costs you nothing; attempt them only if the core is done well.

- Warn before the author navigates away from unsaved changes.
- Move a reference to a new position or level.
- Delete a topic, with a clear rule for topics that books still reference.
- Front-end tests.
- A single-command start, for example with Docker Compose.

What we look for

AREA	WHAT WE WANT TO SEE
Domain modelling	The data model reflects the reference-based structure. Reuse is a first-class concern.
API design	Endpoints are RESTful and intuitive. HTTP verbs, status codes and error responses are used correctly and consistently.
Front-end architecture	Components, state and data fetching are organised so another developer could extend them. Server state stays in sync after edits.
Code quality	Readable, consistent and appropriately structured. We prefer clarity over cleverness.
Testing	A few well-chosen tests on the behaviour that matters.
Communication	Trade-offs are noted. Another developer could pick the project up from your README.

What we are not looking for

- A production-ready system. In-memory storage is fine.
- Perfect coverage of every edge case.

- Visual polish or mobile layouts.
- A specific framework, language or architecture. Use what you know best.

Submission

Send us a link to your repository, or a ZIP of your project. Include a short README covering:

- How to run the project, including ports.
- Your design decisions and trade-offs, including the status codes you chose where more than one was defensible.
- What happens to indented entries when a reference above them is removed.
- How you would handle deleting a topic that books still reference.
- What is unfinished, and what you would do differently with more time.

> NEXT STEP
>
> After we review your submission, we will book a follow-up conversation of about an hour. You will walk us through your design, and we will make a small change to it together.